query causing a full-table scan can be a disaster in such a case; however, a well-indexed table performs pretty well. Compressed tables are a great utility when distributing a database on space-constrained media like CDs.

- MyISAM permits NULL values in indexed columns.
- Each character (CHAR/VARCHAR) column can have a different character set. For instance, in one column you may store ASCII character values, and in the other you may store UTF-8 or UTF-16 values.

Understanding the B-tree

Anybody dealing with a respectable database-management system must have heard the term 'B-tree'. Some systems and databases also use the notion of B+tree. What is it and how does it fit into the picture of database-management systems? Here is what Wikipedia says on the topic:

"In computer science, a B-tree is a tree data structure that keeps data sorted and allows searches, sequential access, insertions, and deletions in logarithmic amortised time. The B-tree is a generalisation of a binary search tree, in that a node can have more than two children. Unlike self-balancing binary search trees, the B-tree is optimised for systems that read and write large blocks of data. It is commonly used in databases and file-systems. Figure 1 shows a very simple B-tree. Notice the difference between conventional binary search trees and a B-tree. A B-tree is not 'binary', that is, each node having 2 child nodes. Essentially, a B-tree allows storing data in a manner so that searching records, inserting new ones, and deleting existing ones is very fast.

Now, let's come back to the question about what the relationship between MyISAM and a B-tree is, and how you can write better queries by getting details on this subject.

The B-tree index

When you create an index in a MyISAM table, its type is B-tree. Figure 2 shows how MyISAM stores the index(es) in the .MYI file and data in the .MYD file. Don't be bothered if this appears a bit daunting. The following is a description of how data and indexes are stored in case of B-tree indexes, in MyISAM tables:

I. On top, we have the root node of the B-tree. This is not specific to MySQL or MyISAM. Every tree has to have a top-level/root node. Traversal starts from this node in most cases. In B-tree terminology, this is called a non-leaf node. Leaf nodes are those that have no child nodes.

II. Below the root node, we have leaf nodes that are connected with the root node. The link between the root node and the immediate children is shown as 'pointers'; however, it is not necessary that these be C/C++ style pointers.

III. Figure 2 shows a B-tree with two levels; however, in a real case, the tree may be much more complex, with multiple levels. This representation is to give you a basic idea about how B-tree works in the case of MyISAM.

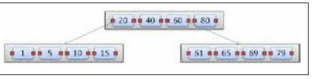

Figure 1: Pictorial representation of a B-tree

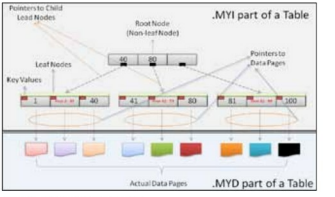

Figure 2: A B-tree index in MyISAM

IV. In this figure, three leaf nodes are shown. Each leaf node holds the keys of a different range: 1 to 40, 41 to 80, and 81 to 100. Please note that the set of ranges is purely arbitrary, and meant purely for discussion purposes. This has no connection with how MyISAM breaks down the B-tree.

V. Below the leaf nodes are the actual data pages—i.e., the table data. These are connected with leaf nodes on the basis of key values. This is how searches on a well-indexed table get results at lightning speed.

VI. Please note that the link between leaf nodes and data pages, which is marked as 'Pointer to Data Pages' isn't a C/C++ style pointer. Instead, leaf nodes hold enough information required to access the exact position of a row (or rows) in the .MYD file. In other words, data can be read/updated/deleted with the least required file I/O.

VII. To make this a little simpler, let's take an example query: *SELECT * FROM SOME_TABLE WHERE SOME_COLUMN = 15;*

a. It is assumed that the column *SOME_COLUMN* is indexed.
b. From the root node, follow the left channel. This is based on simple maths, 15 < 40. This lands you at the left-most leaf node.
c. In the left node, you will find the key with the value 15.
d. Once you find the key, you'll know if there is any data in the .MYD file with *SOME_COLUMN = 15*.
e. If so, retrieve the information required to reach that exact location in the .MYD file, to read the row in question.

After a close look at Figure 2, you can answer a question that is often a matter of debate—or rather, confusion. Does MyISAM support clustered indexes? The answer is, " NO!" In case of a clustered index, data is stored in a sorted order,